

PROJECT DOCUMENTATION

 Project Title: TruthLens – AI vs Human Text Detection System

① Project Overview

HumanizeAI is a mobile-based intelligent system that detects whether a given text is written by a human or generated by AI.

The application allows users to capture on-screen content, extract text, and analyze it using NLP-based techniques.

② Problem Statement

With the rise of AI tools like ChatGPT, AI-generated content is becoming very common. This creates problems such as:

Difficulty in identifying original vs AI-generated content

Academic dishonesty (assignments, essays)

Misinformation in blogs/news

Lack of transparency in content creation

 Existing tools:

Are paid

Not easily accessible on mobile

Lack real-time detection

③ Proposed Solution

We propose a mobile application with real-time AI detection capability.

Key Idea:

Floating overlay captures screen

OCR extracts text

NLP engine analyzes text

Outputs AI vs Human percentage

④ Objectives

Detect AI-generated content using NLP techniques

Provide real-time analysis on mobile

Build a hybrid system (Mobile + Backend + AI)

Create a scalable and extensible architecture

5 🧩 Key Features

📱 User Features

- Activate floating overlay
- Capture screen content
- Select text from screenshot
- Analyze text
- View AI vs Human percentage

🧠 AI Features

- Perplexity-based analysis
- Burstiness calculation
- Vocabulary richness evaluation
- Repetition detection

📊 Additional Features

- Result visualization (graphs/percentages)
- Explanation of results
- History of previous analyses (optional)

6 🏗️ System Architecture

Android App



Spring Boot Backend



Python NLP Engine



Scoring System



Result → Mobile App

7 Workflow

1. User activates overlay
2. Opens any app (browser/blog)
3. Clicks floating button
4. Screenshot is captured
5. OCR extracts text
6. Text sent to backend
7. NLP model analyzes
8. Result displayed

8 Core Algorithm

Features Used:

Feature	Purpose
Perplexity	Measures predictability
Burstiness	Measures sentence variation
Vocabulary Richness	Measures diversity
Repetition Score	Detects redundancy

Final Score Calculation:

Final Score =
 $0.4 \times \text{Perplexity} +$
 $0.2 \times \text{Burstiness} +$
 $0.2 \times \text{Vocabulary} +$
 $0.2 \times \text{Repetition}$

9 Tech Stack

 Frontend (Mobile)
Android (Java / Kotlin)
XML UI
Google ML Kit (OCR)

 Backend
Spring Boot (Java)
REST APIs

 NLP Engine
Python
Transformers (GPT2)
NLTK
NumPy

 Database (Optional)
MongoDB / SQLite

10 Project Structure

```
project/
├── android-app/
│   ├── ui/
│   ├── overlay/
│   └── api/
├── springboot-backend/
│   ├── controller/
│   ├── service/
│   └── model/
├── python-nlp/
│   ├── main.py
│   └── analyzer.py
```

17 Development Timeline (Weekly Plan)

Week 1:

Project planning

UI design (basic screens)

Week 2:

Android app basic setup

Implement home screen

Week 3:

Floating overlay implementation

Screen capture functionality

Week 4:

OCR integration (text extraction)

Week 5:

Python NLP model (perplexity + burstiness)

Week 6:

Spring Boot backend setup

API creation

Week 7:

Integration (Android ↔ Backend ↔ NLP)

Week 8:

Testing + debugging

UI improvements

12 ⚠ Challenges Faced

- Handling floating overlay permissions
- Accurate OCR extraction
- Implementing perplexity efficiently
- Performance optimization (slow NLP models)
- Lack of 100% accuracy in AI detection

13 🚀 Future Enhancements

- Chrome extension version
- Real-time text detection (without screenshot)
- Multi-language support
- Improved ML model (fine-tuned)
- Cloud deployment
- Sentence-level highlighting

14 📈 Advantages

- Fully custom AI detection system
- No dependency on paid APIs
- Real-time mobile usability
- Scalable architecture

15 ⚠ Limitations

- Not 100% accurate
- Performance depends on device/server
- Requires internet for backend

-----_-----++++-+-----+++++++--

Week plan

⚡ 1-WEEK COMPLETE PLAN (WITH DEPLOYMENT)

Goal:

Build + integrate + deploy:

Android App 

Spring Boot 

Python NLP Engine 

Live API (deployed) 

DAY 1 – Setup & Planning

Tasks:

Finalize project name

Install:

Android Studio

Java + Maven

Python

Create structure:

android-app/

springboot-backend/

python-nlp/

—

DAY 2 – Python NLP Engine (Core + Accuracy 🔥)

Tasks:

Install:

transformers

NLTK

Implement:

Perplexity

Burstiness

Vocabulary richness

Repetition

✓ Improve Accuracy:
Normalize values
Limit input length (300–500 words)
Weighted scoring model

DAY 3 – Explanation Engine

✓ Tasks:
Add explanation logic:
Low perplexity → AI-like
Low burstiness → uniform text
High repetition → AI

✓ Output:
{
 "AI": 72,
 "Human": 28,
 "explanation": ["Low perplexity", "Low variation"]
}

DAY 4 – Spring Boot Backend

✓ Tasks:
Create API:
POST /analyze
Connect: 🤝 Spring Boot → Python API
Test using Postman

DAY 5 – Android App + API

✓ Tasks:
Build UI:
Home screen
Result screen
Connect: 🤝 Android → Spring Boot

✓ Show:
AI %
Human %
Explanation

📅 DAY 6 – Floating Button + OCR 🔥

✓ Tasks:
Add overlay permission
Create floating draggable button

✓ On click:
Capture screen
Extract text using
→ Google ML Kit

⚠️ Test on real device

📅 DAY 7 – 🚀 DEPLOYMENT + FINAL INTEGRATION

🔥 Step 1: Deploy Python NLP API

Options:

Render (easy)

Railway

VPS

👉 Recommended:

Render

🔥 Step 2: Deploy Spring Boot

Options:

Render

Railway

AWS (advanced)

👉 Use same platform for both

🔥 Step 3: Update URLs

Replace:

`http://localhost:8080`

With:

`https://your-api-url.com`

🔥 Step 4: Test Live System

Mobile → Live API → Result

🔥 Step 5: Final Polish

UI improvements

Loading indicator

Error handling